MIT 8103 Advanced Database Systems: Individual Practical Database Portfolio

Student: Oluwakeyede Odutayo **Student ID:** 301907379
Miva email: o.odutayo7379@miva.edu.ng
GitHub repository: https://github.com/Oluwakeyede/advanced-database-portfolio

Selected case study

Organisation: West Haven Hotel and Suites, a small hotel. The way it operates is real; all data in the portfolio is simulated.
System: a hotel reservation and front-desk system.

Users: front-desk staff, the manager and the accountant.

Major data: guests, room types and rates, rooms, bookings, the rooms in each booking, payments, and front-desk staff and shifts.

Key operations: reserve one or more rooms, check availability, cancel, check in and out, record payments, and report daily revenue and occupancy. The same case study is used in all five portfolios.

Portfolio 1: Database design and modelling

I designed eight MySQL 8.0 tables: guests, room_types, rooms, staff, staff_shifts, bookings, booking_rooms and payments. A booking can include several rooms, so the many-to-many link is resolved with booking_rooms, which also stores rate_charged so later rate changes do not alter old bookings. Primary and foreign keys connect the tables, and UNIQUE, NOT NULL and CHECK constraints reject bad data (for example check-out must be after check-in, amounts must be above zero, and each shift date has one staff member). The design meets 3NF: rooms are not stored as a list inside bookings (1NF), guest details live only in guests (2NF), and room type name and rate live only in room_types (3NF). Evidence in the repository: schema.sql, seed_data.sql, ER diagram and schema screenshot.

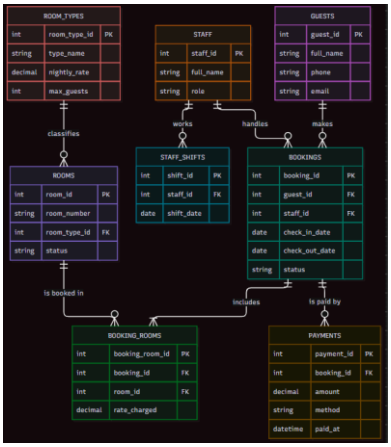


Figure 1. ER diagram

Portfolio 2: Query processing and optimisation

Three queries ran against the 40 simulated bookings: bookings with guest details (join), total payments per booking (LEFT JOIN with SUM) and bookings per status (GROUP BY). Results are exported as CSV files. EXPLAIN on a guest-name search showed access type ALL with 40 rows estimated before indexing. After creating idx_guests_full_name, it showed type ref with 1 row (Oracle Corporation, n.d.). The rows value is the optimiser's estimate, not a measured run time, so I claim a better access plan rather than a measured speed-up. Indexes use storage and slow writes, so they should follow real query patterns.

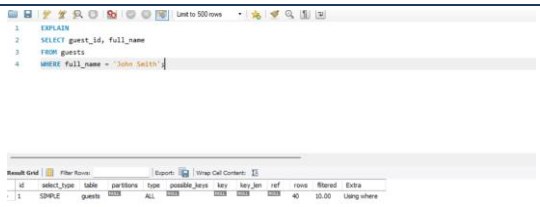


Figure 2a. EXPLAIN before index

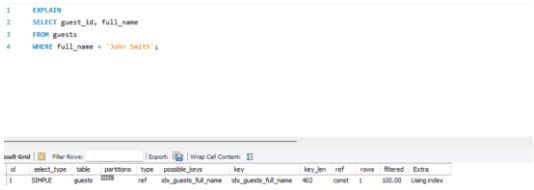


Figure 2b. EXPLAIN after index

Portfolio 3: Transactions and concurrency

Using booking 1 in InnoDB, COMMIT kept a status change and ROLLBACK restored the original status. With two Workbench connections at REPEATABLE READ, Connection B did not see Connection A's uncommitted update and saw it only after A committed. When both connections updated the same row, B's UPDATE waited until A committed, which shows InnoDB row-level locking at work (Oracle Corporation, n.d.). The repository holds the three SQL scripts and screenshots from both connections.

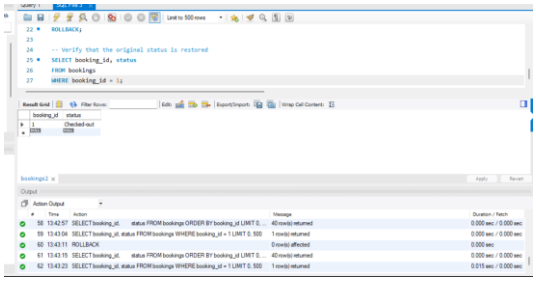


Figure 3a. ROLLBACK restores status

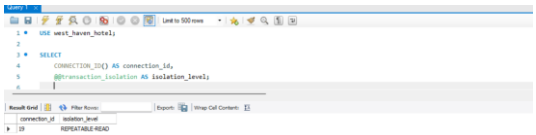


Figure 3b. Connection B isolation level

Portfolio 4: NoSQL and advanced data models

The same data became one MongoDB document per booking, embedding the guest, booking dates and status, staff, a rooms array and a payments array. A Python script (import_bookings.py) read the MySQL seed data and loaded 40 documents into the bookings collection on MongoDB Atlas. Further scripts show CRUD operations (on a separate test collection), queries with dot notation and $gt, three aggregation pipelines ($unwind, $group, $sum, $sort) and a unique index on booking_id (MongoDB, Inc., n.d.). Embedding lets one query return a whole booking, but it duplicates guest and room details, so updates need care.

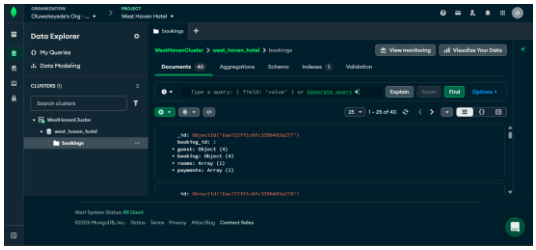


Figure 4a. 40 documents in Atlas

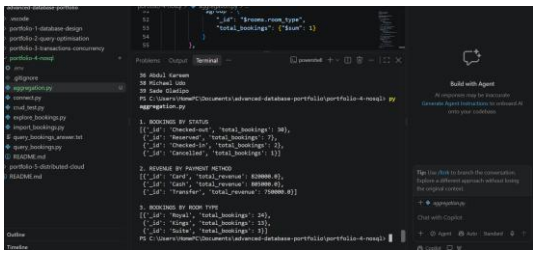


Figure 4b. Aggregation output

Portfolio 5: Distributed and cloud database

The Atlas cluster WestHavenCluster holds the 40 booking documents migrated during Portfolio 4. I had previously connected successfully after resolving an IP access-list issue. When I returned to continue with the practical activities in Portfolio 5, I encountered another connectivity problem. Due to network difficulties and the time available, I could not proceed with practical testing of replication and sharding. I went on to research and explain how replica sets, consistency, failover and sharding work in MongoDB Atlas. These concepts were not practically tested on my cluster (MongoDB, Inc., n.d.).

Individual technical reflection

My main design choice was to resolve bookings and rooms through a link table and to keep rate_charged on it, which kept the data in 3NF without losing price history. The hardest problems were practical. The mysql command was not recognised in PowerShell, so I used MySQL Workbench. The first Atlas connection failed during the TLS handshake, and the cause was that my IP address was not on Atlas's allow list. I also had to correct a wrong Git commit message with an amend. I learned that EXPLAIN shows the planned access path and not run time, that locks make a second writer wait instead of overwriting, and that embedding in MongoDB trades duplicated data for simpler reads. With more time I would test replica-set failover on Atlas and measure query times with EXPLAIN ANALYZE.

AI use declaration

Tools: Claude (Anthropic) & ChatGpt (OpenAI)

Used for: explaining concepts (normalisation, transactions, MongoDB), helping draft parts of the README files and this document, helping write and debug the Python scripts, and troubleshooting Git, MySQL Workbench and Atlas connection errors.

Checked by me: I ran every SQL statement and Python script myself, compared the outputs with my screenshots, corrected errors, edited the text, and can explain the submitted work.

References

MongoDB, Inc. (n.d.). MongoDB manual. https://www.mongodb.com/docs/manual/

Oracle Corporation. (n.d.). MySQL 8.0 reference manual. https://dev.mysql.com/doc/refman/8.0/en/

Miva Open University. (2026). MIT 8103 Advanced Database Systems: Individual practical database portfolio, September cohort 2026 [Assessment brief].